

# VoltTech Store

## Documentación Técnica

---

Universidad Mariano Gálvez  
Ingeniería en Sistemas de Información  
Fecha: Junio 2026  
Versión: 1.0

VoltTech Store es una aplicación web de comercio electrónico desarrollada con React, TypeScript y Supabase. Permite la gestión de un catálogo de productos, carrito de compras, checkout con simulación de pago y generación de facturas electrónicas (FEL). Incluye un panel administrativo completo con gestión de productos, categorías, pedidos y usuarios.

# 1. Stack Tecnológico

---

## Frontend

| Tecnología          | Versión | Uso                                    |
|---------------------|---------|----------------------------------------|
| React               | 19.x    | Biblioteca principal de UI             |
| TypeScript          | 5.x     | Tipado estático                        |
| Vite                | 7.x     | Bundler y servidor de desarrollo       |
| React Router        | 7.x     | Enrutamiento SPA                       |
| TanStack Query      | 5.x     | Fetching y caché de datos del servidor |
| Tailwind CSS        | 4.x     | Estilos utilitarios                    |
| shadcn/ui           | —       | Componentes de interfaz (Radix UI)     |
| @react-pdf/renderer | 4.x     | Generación de PDFs en el navegador     |
| Sonner              | 2.x     | Notificaciones toast                   |

## Backend y Base de datos

| Tecnología         | Versión | Uso                                     |
|--------------------|---------|-----------------------------------------|
| Supabase           | 2.x SDK | Backend-as-a-Service (BaaS)             |
| PostgreSQL         | 15.x    | Base de datos relacional (vía Supabase) |
| Row Level Security | —       | Control de acceso a datos por fila      |
| Supabase Auth      | —       | Autenticación (email + OAuth)           |

## Testing y Calidad

| Herramienta | Uso                                          |
|-------------|----------------------------------------------|
| Vitest      | Framework de pruebas unitarias (Vite-nativo) |
| ESLint      | Análisis estático de código                  |
| Prettier    | Formato de código                            |

## 2. Arquitectura del Sistema

---

VoltTech Store es una Single Page Application (SPA) que se comunica directamente con Supabase mediante su SDK de JavaScript. No existe un servidor backend intermedio; toda la lógica de negocio crítica reside en funciones PostgreSQL (RPCs) ejecutadas dentro de la base de datos.

### Flujo de datos

```
Navegador (React SPA)
• supabase-js SDK (HTTPS)
Supabase Platform
Auth (JWT, sesiones)
PostgREST (CRUD sobre tablas)
RPC (funciones PL/pgSQL)
checkout_order() – transacción atómica
```

### Patrones de acceso a datos

- TanStack Query gestiona todo el fetching, caché e invalidación de queries.
- Las mutaciones críticas (checkout) usan `supabase.rpc()` para ejecutar funciones server-side.
- Row Level Security (RLS) en Supabase garantiza que cada usuario solo acceda a sus propios datos.
- Los administradores se identifican via la tabla `user_roles` con rol "admin".

### Estructura de carpetas

```
src/
components/      Componentes reutilizables
  ui/            Componentes shadcn/ui (Radix)
  FelInvoicePDF  Documento PDF de factura FEL
  Navbar         Barra de navegación
  PaymentDialog  Modal de pago y datos FEL
pages/           Vistas por ruta
  Admin          Panel administrativo
  Cart           Carrito de compras
  Home           Catálogo de productos
  Orders         Historial de pedidos
  ProductDetail  Detalle de producto
  ThankYouPage   Confirmación de compra
lib/
  auth.tsx       Contexto de autenticación
  checkout-utils Funciones puras (testadas)
  utils.ts       Utilidades generales
integrations/supabase/
client.ts        Instancia del cliente Supabase
types.ts         Tipos generados del esquema DB
```

### 3. Esquema de Base de Datos

---

**Tabla: products**

| Columna     | Tipo        | Descripción                |
|-------------|-------------|----------------------------|
| id          | uuid PK     | Identificador único        |
| name        | text        | Nombre del producto        |
| slug        | text UNIQUE | URL amigable               |
| description | text        | Descripción del producto   |
| price       | numeric     | Precio en quetzales        |
| stock       | integer     | Unidades disponibles       |
| image_url   | text        | URL de imagen              |
| category_id | uuid FK     | Referencia a categories    |
| is_active   | boolean     | Si el producto está activo |

**Tabla: orders**

| Columna          | Tipo    | Descripción                                      |
|------------------|---------|--------------------------------------------------|
| id               | uuid PK | Identificador único                              |
| user_id          | uuid FK | Usuario que realizó el pedido                    |
| total            | numeric | Monto total en quetzales                         |
| shipping_address | text    | Dirección de envío                               |
| status           | enum    | pending   paid   shipped   delivered   cancelled |
| nit              | text    | NIT del receptor (FEL)                           |
| fiscal_name      | text    | Nombre fiscal del receptor (FEL)                 |
| invoice_number   | text    | Número DTE simulado (ej. DTE-123456)             |
| invoice_auth     | uuid    | UUID de autorización SAT simulado                |

**Tabla: profiles**

| Columna   | Tipo    | Descripción                               |
|-----------|---------|-------------------------------------------|
| id        | uuid PK | Mismo ID que auth.users                   |
| full_name | text    | Nombre completo                           |
| email     | text    | Email (copiado por trigger de auth.users) |
| is_active | boolean | Soft-delete: false = cuenta desactivada   |

#### Otras tablas

- categories — id, name, slug, description
- cart\_items — id, user\_id, product\_id, quantity
- order\_items — id, order\_id, product\_id, product\_name, unit\_price, quantity
-

user\_roles — id, user\_id, role (admin | user)

## 4. Funcionalidades del Sistema

---

### 4.1 Catálogo de productos

- Listado de productos con nombre, precio, imagen y disponibilidad de stock.
- Filtrado por categorías. Navegación a detalle por slug amigable (/products/:slug).
- Productos con stock = 0 muestran etiqueta "Agotado" y no pueden añadirse al carrito.

### 4.2 Carrito de compras

- Añadir, incrementar, decrementar y eliminar productos.
- Cálculo de total en tiempo real. Persistido en la tabla cart\_items.
- Botón "Finalizar pedido" abre el modal de pago.

### 4.3 Checkout y pago simulado

- Modal con datos de facturación FEL (NIT y nombre fiscal) y tarjeta simulada.
- Validación de formato: tarjeta 16 dígitos, expiración MM/AA, CVV 3 dígitos.
- Llama a la RPC checkout\_order que ejecuta todo en una transacción atómica.
- Redirige a /thank-you/:orderId al confirmar el pago.

### 4.4 Factura Electrónica FEL (simulación)

- Generada automáticamente al realizar el checkout.
- Número DTE: DTE-XXXXXX. Número de autorización: UUID generado por PostgreSQL.
- PDF descargable desde la página de gracias y desde "Mis pedidos".
- Incluye: emisor, receptor (NIT + nombre fiscal), detalle de ítems, subtotal, IVA 12% y total.
- Pie de página: "Simulación con fines educativos — Régimen General con IVA".

### 4.5 Panel administrativo (/admin)

- Acceso restringido a usuarios con rol "admin" (validado en useAuth + RLS).
- Pestaña Productos: CRUD completo con imagen, precio, stock y categoría.
- Pestaña Categorías: crear y eliminar categorías.

- Pestaña Pedidos: ver todos los pedidos y cambiar estado.
- Pestaña Usuarios: ver, editar nombre/rol y desactivar cuentas (soft-delete).

#### **4.6 Gestión de usuarios (admin)**

- Listado de perfiles activos con email, nombre, rol y fecha de registro.
- Edición de nombre completo y cambio de rol (admin " user).
- Desactivación con confirmación: sets is\_active = false (reversible desde la DB).
- El admin no puede editar ni desactivar su propia cuenta.

## 5. Función RPC: checkout\_order

Función PL/pgSQL ejecutada dentro de PostgreSQL que garantiza atomicidad total del proceso de compra. Si cualquier paso falla, Postgres revierte la transacción completa.

### Firma

```
checkout_order(  
  p_user_id      uuid,  
  p_items        jsonb,  -- [{product_id, quantity, product_name, unit_price}]  
  p_total        numeric,  
  p_shipping_address text,  
  p_nit          text,  
  p_fiscal_name  text,  
  p_invoice_number text  
)  
RETURNS uuid  -- order_id
```

### Pasos internos (en una sola transacción)

- 1. Verificar stock: para cada ítem, lee products.stock. Si stock < quantity lanza EXCEPTION 'Sin stock: [nombre]'.
- 2. Crear orden: INSERT en orders con status = 'paid', invoice\_auth = gen\_random\_uuid().
- 3. Crear líneas: INSERT en order\_items con todos los productos del carrito.
- 4. Descontar stock: UPDATE products SET stock = stock - quantity para cada ítem.
- 5. Limpiar carrito: DELETE FROM cart\_items WHERE user\_id = p\_user\_id.
- 6. Retorna el UUID del nuevo pedido.

### Trigger: sync\_user\_email

Trigger AFTER INSERT ON auth.users que copia el email del usuario recién registrado a la tabla profiles. Esto permite al panel admin listar emails sin necesitar la service-role key.

```
CREATE TRIGGER on_auth_user_email  
AFTER INSERT ON auth.users  
FOR EACH ROW EXECUTE FUNCTION public.sync_user_email();
```

## 6. Rutas de la Aplicación

| Ruta            | Componente    | Acceso      |
|-----------------|---------------|-------------|
| /               | Home          | Público     |
| /auth           | Auth          | Público     |
| /products/:slug | ProductDetail | Público     |
| /cart           | CartPage      | Autenticado |



|                     |              |                            |
|---------------------|--------------|----------------------------|
| /orders             | OrdersPage   | Autenticado                |
| /thank-you/:orderId | ThankYouPage | Autenticado + fromCheckout |
| /admin              | AdminPage    | Solo administradores       |

## 7. Pruebas Unitarias

Las pruebas se implementaron con Vitest sobre funciones puras extraídas en `src/lib/checkout-utils.ts`. Se ejecutan con: `npm test`

### Módulo probado: checkout-utils.ts

| Función                         | Casos | Qué se verifica                                                            |
|---------------------------------|-------|----------------------------------------------------------------------------|
| <code>genInvoiceNumber()</code> | 2     | Formato DTE-XXXXXX, retorna string                                         |
| <code>fmtCard()</code>          | 4     | Espacios c/4 dígitos, elimina letras, trunca a 16                          |
| <code>fmtExpiry()</code>        | 4     | Formato MM/YY, sin slash con < 3 dígitos                                   |
| <code>isValidCard()</code>      | 5     | Acepta 16 dígitos, rechaza 15 o 17, acepta con espacios                    |
| <code>isValidExpiry()</code>    | 4     | Meses 01–12, rechaza 00 y 13, sin slash inválido                           |
| <code>isValidCvv()</code>       | 4     | Exactamente 3 dígitos, rechaza letras                                      |
| <code>calcIva()</code>          | 3     | <code>subtotal + iva = total</code> , IVA = 12%, <code>total=0</code> da 0 |
| <code>slugify()</code>          | 4     | Minúsculas, guiones, sin especiales, sin guiones extremos                  |

Total: 30 pruebas — 30 pasando

### Ejecución

```
$ npm test
RUN v4.1.8
Test Files  1 passed (1)
Tests      30 passed (30)
Duration   ~500ms
```

## 8. Instalación y Ejecución

### Requisitos

- Node.js 18+ y npm
- Cuenta en Supabase con el proyecto configurado
- Variables de entorno en `.env` (`VITE_SUPABASE_URL`, `VITE_SUPABASE_PUBLISHABLE_KEY`)

### Comandos

```
npm install      # Instalar dependencias
npm run dev      # Servidor de desarrollo (puerto 8080)
npm run build    # Build de producción
npm test         # Ejecutar pruebas unitarias
npm run lint     # Análisis estático de código
```

### Migración de base de datos

Ejecutar el archivo `supabase/migrations/20260608_fel_users.sql` en el SQL Editor de Supabase para crear las columnas FEL, el trigger de email y la función `checkout_order`.